FLASKGUARD

A Custom Web Application Firewall & Reverse Proxy

Defense de Projet de Fin d'Etudes (Soutenance) | Juin 2026

DEVELOPMENT TEAM:

Amine Saoud (Flask Scaffold & Dashboard UI)

Saaf Ayyoub (YAML Configuration, Rate Limiting, Auth, Regex, Docker)

Abdelmoghit El Asraoui (ML Classifier & IP strikes/bans)

Mohamed Ben Mouh (7-Layer File Scanner & TLS Termination)

Hamza Amhidi (Reverse Proxy, Demo Backend & Tests)

Project Overview & Vision

FlaskGuard is an educational and modular Web Application Firewall designed to run inside Python environment and intercept all incoming HTTP requests. It protects backends from common security exploits using high-speed matching rules combined with machine learning fallback models.

- Lightweight middleware pattern directly integrated into request lifecycle
- Dual-engine request detection: deterministic regex rules + LightGBM statistical ML classifier
- Protects files, forms, paths, query inputs, and HTTP headers
- Provides visibility through a clean administrative operations dashboard

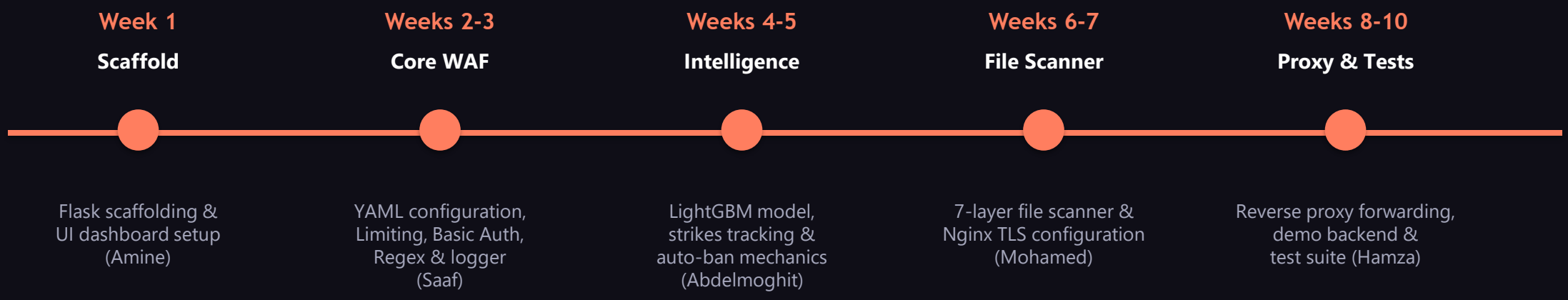
TRAFFIC FLOW BOUNDARY

1. Client makes an HTTPS request to Nginx TLS proxy
2. Proxy terminates TLS, forwards plain HTTP internally
3. FlaskGuard middleware intercepts the request
4. Sequential inspections (auth, rate, WAF checks)
5. Bad traffic blocked with 403; clean traffic proxy-forwarded
6. Target backend receives and replies to proxy transparently

Team Scopes and Division of Responsibilities

Team Member	Project Scope / Owned Components	Key Deliverables
Amine Saoud	Flask application scaffold, request hooks definition, dashboard UI templates (vanilla JS, HTML, CSS)	app.py, dashboard.html
Saaf Ayyoub	YAML configuration loading, rate limiter logic, administrative auth gates, dashboard backend APIs, regex engine, attack loggers, Docker environment	security.py, attack_logger.py, Dockerfile
Abdelmoghit El Asraoui	LightGBM character n-gram classifier training, retraining pipelines, semantic fallback engine, strike tracking, auto-bans, and blacklist	ml_classifier.py, ip_manager.py
Mohamed Ben Mouh	7-layer file scanner pipeline (magic bytes, extension matches, SVGs, archives, macro documents), TLS configuration, ProxyFix middleware	file_scanner.py, Nginx configs
Hamza Amhidi	HTTP reverse proxy routing pipeline, hop-by-hop header stripping, redirect forwarding, demo backend, unit and integration test suite	proxy.py, test_files, tests

Project Timeline and Key Milestones



Development Phase Dependencies

Note: Phase 1 establishes the scaffold which underpins all detection phases. Phase 5 reverse proxies clean requests and tests the entire system.

Multi-Layer Security Pipeline flow

Traffic enters through Nginx (TLS Termination) which proxies the request to the Flask WAF context. The request is evaluated through lightweight checks (Rate limiting and Blacklist verification) before executing deep inspection on file uploads (7-layer scanning), form strings (deterministic regular expression), and statistical models (LightGBM character n-grams). Approved traffic passes through to the backend.

Phase 1 Objectives & Deliverables

OWNER: AMINE SAOUD

- Create the Python package structure and application entry point (flaskguard/app.py)
- Implement the standard before_request and after_request routing hooks to build the firewall execution pipeline
- Integrate flask-limiter scaffolding with custom storage backends to enforce request throttling
- Design a custom, modern admin security dashboard (HTML, CSS, JavaScript) from scratch without external build tools
- Protect administrative stats and control APIs (/admin/*) using HTTP Basic Authentication gates
- Provide mock simulation methods to feed active statistics into the UI for testing

Flask Framework Rationale & Scaffold Architecture

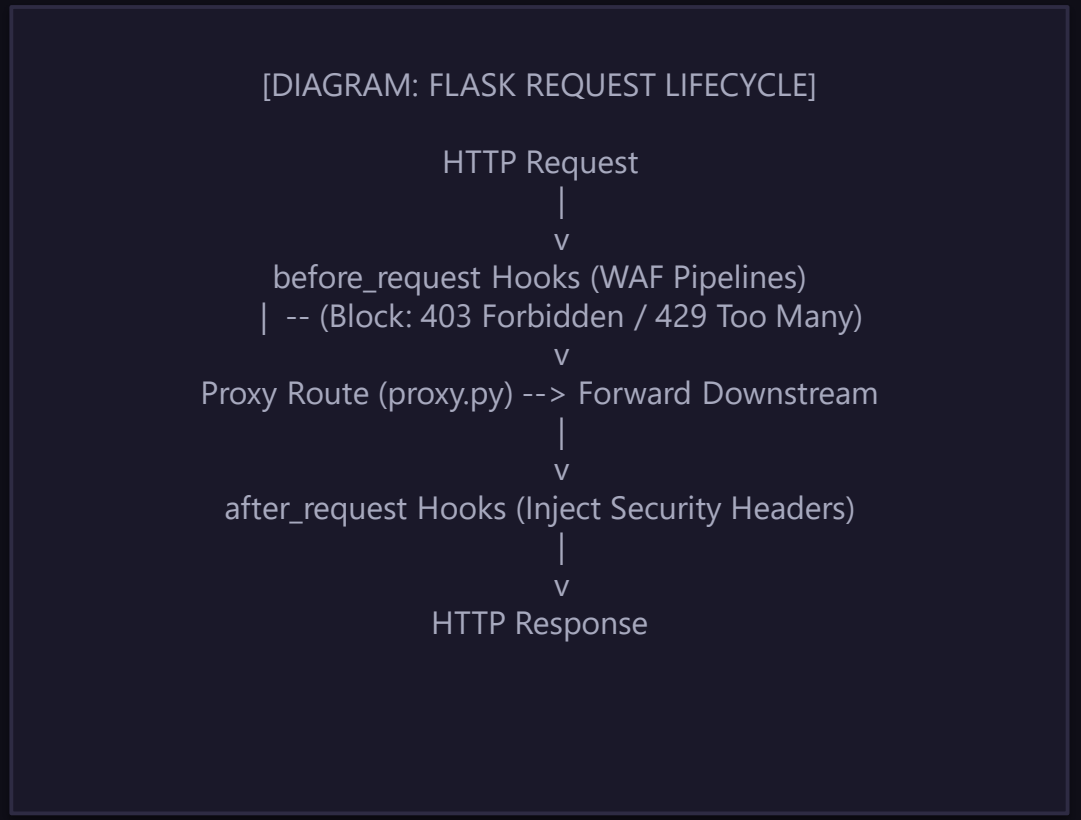
Why Flask was selected for the WAF Middleware:

• Minimal surface area: Core footprint contains only routing, requests, and WSGI adapters, keeping CPU overhead minimal.

• Middleware-friendly hooks: Flask's request lifecycle hooks (before_request, after_request) map naturally to a security inspection pipeline.

• WSGI Compatibility: Serves cleanly on production engines (Gunicorn, Waitress) without framework wrappers.

• Ecosystem Maturity: Simplifies integration of rate limiting, PyYAML configuration parsing, and ML inference libraries.



Project Directory Structure

Directory Layout:

The package layout follows modern Python packaging guidelines. The WAF core is housed within the 'flaskguard' package directory, keeping components modular and decoupled from the target backend application.

```
FlaskGuard/
├── flaskguard/
│   ├── app.py                # Main Flask app (entry point,
routes, pipeline)
│   ├── security.py           # Regex attack detection rules
│   ├── ml_classifier.py      # LightGBM classifier pipeline
│   ├── file_scanner.py       # 7-layer file upload scanner
│   ├── ip_manager.py         # IP strikes and blacklist tracking
│   ├── attack_logger.py      # Structured file logging (JSON +
text)
│   ├── proxy.py              # HTTP transparent reverse proxy
forwarding
│   └── templates/
│       └── dashboard.html    # Custom SPA Admin dashboard UI
├── config.yml                # Main configuration parameters
├── Dockerfile                # WAF container image
└── docker-compose.yml        # Multi-container network
composition
```

Dashboard UI Design & Tech Stack

- Vanilla Tech Stack: Implemented strictly using HTML5, CSS3, and modern ES6 JavaScript to guarantee zero-build deployment (no npm webpack/vite toolchains required).
- Responsive Grid Layout: Styles are structured using pure CSS Grid and Flexbox, adjusting seamlessly across mobile, tablet, and desktop monitors.
- Single-Page Application (SPA): Interactivity is handled via asynchronous JS fetch() calls, updating statistics, tables, and blacklist status without full-page reloads.
- Secure Administrative Gate: Restricts access to the /admin route by enforcing Basic Authentication before rendering templates.
- Self-Contained Codebase: Served directly from Flask's /templates and /static directories, eliminating external CDN dependencies to secure offline deployments.

Dashboard UI Visual Layout and Statistics

Total Attacks	Blocked Today	Active Bans	Avg Latency
1,247	89	12	23ms

RECENT SECURITY EVENTS (LOGS)

```
2026-06-12 23:42:01 | IP: 198.51.100.12  | SQL_INJECTION   | Payload: ' OR 1=1 -- | regex
2026-06-12 23:41:45 | IP: 203.0.113.88   | XSS_ATTACK      | Payload: <script>...  | regex
2026-06-12 23:40:12 | IP: 198.51.100.12  | MALICIOUS_FILE  | File: malware.exe     | scanner
2026-06-12 23:39:03 | IP: 192.0.2.14     | PATH_TRAVERSAL  | Payload: ../../etc/   | regex
```

ACTIVE IP BANS

198.51.100.12 (Strikes: 5, expires in 45m)

203.0.113.250 (Permanent ban, manual)

198.51.100.77 (Strikes: 5, expires in 58m)

Phase 2 Objectives & Deliverables

OWNER: SAAF AYYOUB

- Develop the YAML configuration subsystem (config.py/config.yml) supporting environment variable overrides
- Implement rate limiting logic based on IP address using the flask-limiter module
- Secure the /admin namespace with HTTP Basic Authentication gates
- Write rest API endpoints (/admin/api/*) to expose WAF stats, attack logs, and blacklist controls to the UI
- Construct the regular expression engine containing 30+ rules for SQLi, XSS, Path Traversal, and Command Injection
- Build the structured logging subsystem generating both attacks.json and plaintext security.log files
- Containerize the entire application topology using Docker and Docker Compose

YAML Configuration & Environment Overrides

Twelve-Factor App Configuration:

• Centralized config: The WAF behavior is configured via config.yml, isolating paths, rate limits, rules, and secrets.

• Environment overrides: Environment variables can override any config value, avoiding baking credentials or local paths into container images.

• Strict fallback typecasting: Numeric configuration parameters (such as ports or thresholds) are dynamically typecast upon override loading.

```yaml
# config.yml snippet
admin:
  username: "admin"
  password: "WafGuard2026"

rate_limiting:
  enabled: true
  limits:
    - "200 per day"
    - "100 per hour"

ip_management:
  strike_threshold: 5
  ban_duration_minutes: 60

proxy:
  backend_url: "http://backend:8000"
```

Rate Limiting & Admin Authentication gates

IP-BASED RATE LIMITING

- Enforces tiered policies per IP (e.g., 200 requests/day, 100 requests/hour)
- Leverages flask-limiter module bound to Flask's request pipeline
- Uses customizable storage backends (memory, redis)
- Returns HTTP 429 Too Many Requests response automatically on violation

HTTP BASIC AUTH GATES

- Secures all administrative endpoints (/admin and /admin/api/*)
- Requires username/password credentials transmitted via Authorization header
- Returns HTTP 401 Unauthorized with standard 'WWW-Authenticate' challenges
- Configured securely via config overrides (prevents default credentials leak)

Regex-Based Attack Detection Engine

Regex Engine Architecture:

• Custom Normalisation: Decodes URL encodings and HTML entities, converts payloads to lowercase, and strips comments to prevent basic evasion.

• 30+ Compiled Rules: Scans query parameters, forms, and request bodies against patterns targeting 5 major categories:

- SQL Injection (SQLi)
- Cross-Site Scripting (XSS)
- Path Traversal
- Command Injection (CMDi)
- File Injection

```
# security.py - Core Ruleset
REGEX_PATTERNS = {
  'SQL_INJECTION': re.compile(

r"(\b(select|union|insert|update|delete|drop|alter|where|from|havin
g|limit)\b)|('|\-\-)#", re.IGNORECASE),
  'XSS_ATTACK': re.compile(
    r"(<script.*?>.*?</script>)|(javascript:)|(\bon[a-z]+\s*=)",
re.IGNORECASE),
  'PATH_TRAVERSAL': re.compile(
    r"(\.\./|\.\.\\)", re.IGNORECASE),
  'COMMAND_INJECTION': re.compile(
    r"(;\s*\b(cat|ls|sh|bash|nc|curl|wget)\b)",
    re.IGNORECASE)
}
```

Structured Logging & Auditing

JSON AUDIT LOG (attacks.json)

Machine-readable logs containing complete metadata, used to render dashboard metrics:

```
{
  "timestamp": "2026-06-09T14:23:07",
  "ip": "203.0.113.42",
  "category": "SQL_INJECTION",
  "method": "regex",
  "path": "/search",
  "payload": "' OR 1=1 --"
}
```

PLAIN SECURITY LOG (security.log)

Plaintext log format optimized for SIEM (Splunk, ELK) ingestion and real-time terminal tailing:

```
2026-06-09 14:23:07 [ATTACK] IP=203.0.113.42
TYPE=SQL_INJECTION METHOD=regex PATH=/search
PAYLOAD="' OR 1=1 --"
```

Docker Containerisation & Deployment Topology

Multi-Container Architecture:

• Layered Isolation: FlaskGuard runs as a reverse proxy shield in front of the target backend, communicating via an isolated Docker bridge network.

• Lightweight Images: Built using python:3.10-slim base image to reduce attack surface and deployment footprint.

• Docker Compose Orchestration: Spins up WAF, front-end TLS proxy, and target backends using a single configuration command.

```
# docker-compose.yml snippet
services:
  flaskguard:
    build: .
    ports:
      - "5000:5000"
    environment:
      - PROXY_BACKEND_URL=http://backend:8000
    networks:
      - waf_net

  backend:
    image: demo_backend
    networks:
      - waf_net
```

Phase 3 Objectives & Deliverables

OWNER: ABDELMOGHIT EL ASRAOUI

- Implement a LightGBM gradient boosting machine learning classifier using character-level TF-IDF features
- Provide a Naive Bayes/Random Forest fallback classification logic when dependencies or computational limits warrant
- Design the training, retraining, and prediction pipeline with dedicated REST endpoints (/ml/status, /ml/retrain)
- Build a semantic fallback system to intercept complex semantic payloads (NoSQL injection, LDAP, template injection)
- Develop the IP strike-tracking manager (ip_manager.py) to log attack frequency per client IP address
- Implement automated ban logic (threshold of 5 strikes triggers a 60-minute ban) and persist to blacklist.json

The Evasion and Repeat Offender Problems

The Evasion Problem:
Deterministic regex engines are highly effective against static patterns but brittle against evasion tricks. Attackers leverage comment insertion, url/case variation, and novel payloads to bypass regular expressions while retaining exploit semantics.

A machine learning model trained on the character statistics of malicious vs benign strings catches variations that bypass regular rules.

THE REPEAT OFFENDER PROBLEM

Allowing attackers to probe endpoints indefinitely increases database load and increases risk. A WAF must recognize behavioral patterns from single hosts.

FlaskGuard introduces strike tracking: every block increments an IP strike count. Crossing the threshold triggers an immediate blacklist ban, dropping connections at the firewall boundary.

LightGBM Classifier & TF-IDF Feature Extraction

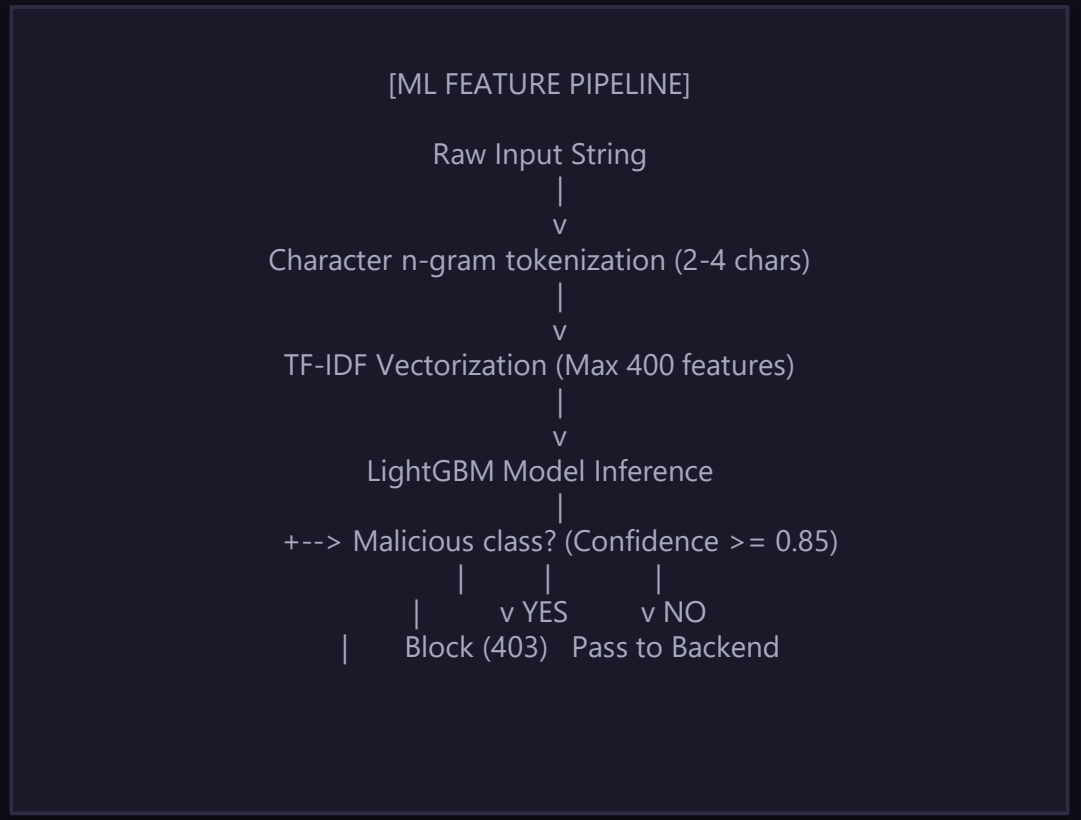
Classification Engine Details:

• Character n-grams (2-4): Converts input strings into statistical combinations of character chunks, catching patterns of scripting syntax.

• TF-IDF Vectorization: Converts text to feature vectors representing importance. Limited to 400 features to control CPU overhead.

• LightGBM Algorithm: Gradient boosting framework utilizing tree-based learning algorithms, offering high throughput and low latency.

• Fallback Classifier: Automatically falls back to Naive Bayes or Random Forest if dependency conflicts occur.



Training pipelines & Feedback Loops

- Dataset details: The default model is trained on a dataset of 630 labeled samples (attacks and benign payloads), split using 5-fold cross-validation.
- Model Persistence: The TF-IDF vectorizer and trained LightGBM model are persisted as joblib serialized files (classifier.pkl) and model metadata (model_meta.json).
- Retraining API: Exposes a POST endpoint (/admin/api/ml/retrain) allowing administrators to trigger model retraining on-demand when new attack patterns are logged.
- Human-in-the-Loop Feedback: Operators can submit prediction feedback (false-positive reports) using the feedback endpoint, appending labeled samples to the active dataset.
- Dynamic Model Versioning: Compares performance metrics of the newly retrained model against the active model before deployment, preventing performance regression.

Semantic Fallback Subsystem

Intercepting Advanced Injections:

Some injection categories do not trigger statistical TF-IDF signatures but rely on logical operators or server-side structural changes.

The Semantic Fallback layer applies targeted, logical heuristics on clean-passing requests to block specific injection styles:

- Server-side Template Injection (SSTI)
- NoSQL Syntax Injection
- LDAP Query Injection

```
# ml_classifier.py - SSTI & NoSQL Heuristics
def _semantic_attack_confidence(payload):
  # SSTI Jinja2/Mako patterns
  if "{{" in payload and "}}" in payload:
    if "config" in payload or "class" in payload:
      return 0.95

  # NoSQL operators
  if "$gt" in payload or "$ne" in payload:
    if "$where" in payload or "$regex" in payload:
      return 0.90

  # LDAP search strings
  if "*" in payload and ("(" in payload or ")" in payload):
    if "objectClass" in payload:
      return 0.88

  return 0.0
```

IP Reputation & Blacklist Logic

Automated Strike Escalation:

• ip_manager.record_strike(ip): Registered in the WAF attack logging block. Increments the strike count of the attacking source IP.

• Auto-Ban Threshold: When the strike count reaches 5, the IP is automatically added to the blacklist.

• Ban Duration: Implements a 60-minute ban window. Connections are rejected at the blacklist hook without reaching regex/ML inspection layers.

• Persistence: The blacklist is stored in blacklist.json, ensuring bans survive container restarts.

BLACKLIST JSON SCHEMA


```
{
  "198.51.100.12": {
    "reason": "Auto-ban: strike threshold exceeded",
    "ban_timestamp": "2026-06-12T23:40:12",
    "expiry": "2026-06-13T00:40:12",
    "strikes": 5
  },
  "203.0.113.250": {
    "reason": "Manual ban by administrator",
    "ban_timestamp": "2026-06-12T20:10:00",
    "expiry": "permanent",
    "strikes": 0
  }
}
```

Phase 4 Objectives & Deliverables

OWNER: MOHAMED BEN MOUH

- Develop the 7-layer file scanner pipeline (file_scanner.py) to inspect multipart upload files in-memory
- Implement Layer 1 filename sanitization and Layer 2 extension blocklisting
- Integrate Layer 3 magic byte detection using python-magic to identify files by contents
- Build Layer 4 content regex scanning and Layer 5 SVG active script injection block
- Add Layer 6 archive bomb detection and Layer 7 Office Open XML macro scanner
- Prevent upload bypasses using double-extension validation (e.g. malaria.php.jpg)
- Configure front-end TLS termination using Nginx and add ProxyFix header middleware

The File Upload Attack Surface

Vulnerabilities in File Uploads:

• Remote Code Execution (RCE): Uploading server-side script files (e.g. PHP webshells) that are executed by the web server.

• Stored Cross-Site Scripting (XSS): Uploading SVG graphics containing embedded Javascript scripts, running inside the visitor's browser.

• Denial of Service (DoS): Uploading archive bombs designed to expand exponentially, consuming disk space and CPU memory.

• Macro Viruses: Delivery of Office documents containing embedded VBA macros designed to execute code on local client systems.

COMMON UPLOAD BYPASS TRICKS

- Falsified Extension: Renaming malware.exe to malware.txt (Bypassed via magic bytes verification)
- Spoofed Content-Type: Header claim of image/jpeg for raw PHP scripts (Bypassed via magic bytes)
- Double Extension: Naming malware.php.jpg to fool suffix filters (Bypassed via multi-dot analysis)
- Null Byte Injection: Truncating names using null bytes (e.g. shell.php\x00.jpg)

The 7-Layer File Scanner Pipeline

Layer 1	**Filename Sanitisation**	Rejects null bytes, path traversal; applies secure_filename()
Layer 2	**Extension Blocklist**	Blocks .php, .exe, .sh, .bat, .asp, etc. suffix matches
Layer 3	**Magic Byte check**	Validates MIME type from file content using python-magic
Layer 4	**Text Content Scan**	Decodes UTF-8 text and matches against WAF regex rules
Layer 5	**SVG Script Block**	XML parses SVG to block <script> and onload attributes
Layer 6	**Archive Bomb Scan**	Checks ZIP metadata for size limits (>50MB) and ratio (>100:1)
Layer 7	**Office Macro Scan**	Scans docx/xlsx structures for vbaProject.bin macros

SVG active Content & Double-Extensions

SVG ACTIVE CONTENT DETECTION

- SVG files are XML-based vector graphics that can legitimately contain scripting elements.
- Browsers will execute these scripts when displaying SVG directly (Stored XSS).
- FlaskGuard parses the SVG XML tree and rejects files if they contain <script> elements.
- Blocks inline event handler attributes (onload, onclick, onerror) on XML elements.
- Blocks href values containing javascript: schemas.

DOUBLE-EXTENSION PROTECTION

- Attackers submit files named malware.php.jpg to trick naive extension matches (which look only at the last dot).
- Web servers might execute the script due to multi-extension handling configurations.
- FlaskGuard performs multi-dot checks: splits filename on all dots and analyzes inner extensions.
- Rejects the file if any inner extension (e.g. 'php' inside malware.php.jpg) matches a blocked script extension.

Archive Bombs & Office Macro Detection

ARCHIVE BOMB DETECTION

- Mitigates Zip Bombs (e.g. 42.zip) that crash processing servers by exhausting memory and disk space.
- FlaskGuard reads zip metadata in-memory using the zipfile library without decompressing.
- Checks cumulative uncompressed sizes of all members.
- Rejects files exceeding 50 MB uncompressed threshold.
- Calculates compression ratio; rejects files if the ratio exceeds 100:1.

OFFICE DOCUMENT MACRO DETECTION

- Microsoft Office files (docx, xlsx, pptx) are zipped XML container archives.
- Attackers embed malicious VBA macros in office documents to compromise clients.
- FlaskGuard decompresses Office documents in-memory.
- Scans for macro indicator files, such as vbaProject.bin or macro sheets.
- Rejects the document if hidden automation code is detected.

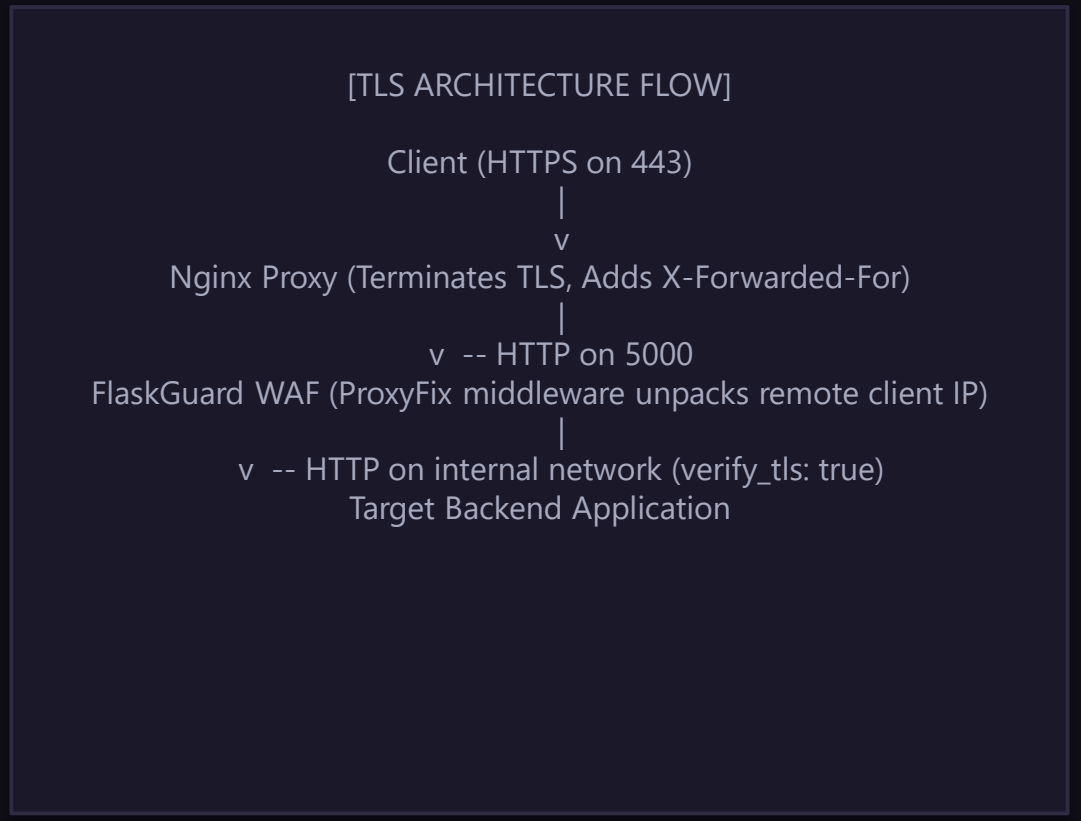
TLS Termination & ProxyFix middleware

Offloading Cryptography to Nginx:

• Front-End TLS termination: Nginx intercepts HTTPS traffic, handling cryptographic overhead, and proxies clean HTTP internally to FlaskGuard.

• Modern Protocols: Disables legacy SSL and TLS 1.0/1.1; supports only secure TLS 1.2 and 1.3 ciphers.

• ProxyFix middleware: Restores client remote IP addresses (replaces proxy IP in request.remote_addr).



Phase 5 Objectives & Deliverables

OWNER: HAMZA AMHIDI

- Implement a transparent HTTP reverse proxy forwarding pipeline (proxy.py)
- Preserve original HTTP method, paths, query variables, cookies, and body stream payloads
- Enforce hop-by-hop header management (strip connection-specific headers before forwarding)
- Handle redirects transparently, forwarding Location headers directly to the client without following them
- Deploy a vulnerable demo backend application containing search SQLi/XSS entry points
- Construct a comprehensive unit and integration test suite to validate all inspection layers

Reverse Proxy Forwarding Mechanics

Transparent Imagine a VIP club. The bouncer (FlaskGuard) checks IDs. If you're on the list, the bouncer doesn't just say "go in" – he personally walks you to the right table and brings back anything the waiter gives you. You never talk directly to the waiter. The bouncer is the middleman.

That's exactly what the reverse proxy does:
Takes your safe request (after all checks passed)
Forwards it to the real website (the "backend")
Gets the website's response
Sends it back to you

Request Forwarding – What gets copied?
HTTP method (GET, POST, etc.) – unchanged.
URL path and query – e.g., /search?q=laptop – unchanged.
Request body – your form data, JSON, file uploads – forwarded exactly as is.
Cookies – so you stay logged in.

```
# proxy.py - Transparent Routing
def forward_request(request, backend_url):
  # Re-build target URL
  url = f"{backend_url}{request.path}"
  if request.query_string:
    url = f"{url}?{request.query_string.decode()}"

  # Forward payload, headers, cookies
  response = requests.request(
    method=request.method,
    url=url,
    headers=clean_headers(request.headers),
    data=request.get_data(),
    cookies=request.cookies,
    allow_redirects=False
  )
  return response
```

Header & Redirect Management

HEADER STRIPPING POLICIES

Some headers are only for the connection between you and the guard. They must be stripped before forwarding to the backend. Examples: Connection, Keep-Alive, Transfer-Encoding – these are like “walkie-talkie instructions” between you and the guard; the backend doesn’t need them.
Other headers are added so the backend knows who the real client is:
X-Forwarded-For – your real IP address.
X-Forwarded-Proto – whether you used HTTPS or HTTP.
Host – rewritten to the backend’s address..

REDIRECT MANAGEMENT

If the backend says “go to this other URL”, the guard does not follow it automatically. Instead, it sends the redirect back to you. Why? Because the new URL also needs to be inspected! If the guard silently followed it, an attacker could bypass the WAF. So you get the redirect, your browser follows it, and the guard checks the new request again. Safer.

Error Handling - What if the backend is down?

If the backend takes too long → 504 Gateway Timeout.
If the backend refuses connection → 502 Bad Gateway.
If the backend returns a 500 error (its own fault) → pass that error back to you unchanged.

Vulnerable Demo Backend Target

What is it?

fake, simple website built just for testing. It's like a dummy store with fake products and a fake search bar.

Why do we need it?

Testing without risk – You can throw attack payloads at it without breaking a real website.

Out-of-the-box demo – Run docker compose up and instantly see FlaskGuard working.

What endpoints does it have?

/	Shows a static home page
/search?q=...	Intentionally vulnerable – it reflects your search term back in the HTML. Without FlaskGuard, you could inject <script>alert(1)</script> and it would run. With FlaskGuard, that payload gets blocked.
/upload	Accepts file uploads (tests the file scanner)
/about	Static page
/api/products	Returns a JSON list of products

```
SIMULATION CURL COMMANDS

# 1. SQLi - BLOCKED (HTTP 403)
curl -I "http://localhost:5000/search?q=' OR 1=1 --"
HTTP/1.1 403 FORBIDDEN

# 2. XSS - BLOCKED (HTTP 403)
curl -I
"http://localhost:5000/search?q=<script>alert(1)</script>"
HTTP/1.1 403 FORBIDDEN

# 3. Normal request - FORWARDED (HTTP 200)
curl -I "http://localhost:5000/search?q=laptop"
HTTP/1.1 200 OK
```

WAF Test Suite & Verification Results

Why ? ?

Imagine you change one rule in the regex engine. How do you know you didn't break file scanning? The test suite runs hundreds of automated checks every time you change code.

Types of tests:

Detection rules – Does ' OR 1=1 -- correctly trigger SQL injection? Does a normal search like "cat food" not trigger it?

ml classifier : verify the ml works.

File scanner layers – Does a .jpg that's actually a PHP script get blocked? Does a real JPEG pass?

IP manager – If I record 5 strikes, does the IP get banned? Does the ban expire after 60 minutes?

Full Request Processing Lifecycle

1. Client HTTP request arrives at the front-end interface.

2. Rate limiter evaluates the client IP. If limits are exceeded, blocks request (HTTP 429).

3. Blacklist checker evaluates source IP. Banned IPs are blocked immediately (HTTP 403).

4. If request is a file upload, it runs through the 7-layer file scanner pipeline. Rejections return HTTP 403.

5. The request fields are scanned against the compiled regular expression rules. Matches return HTTP 403.

6. If regex checks pass, the LightGBM classifier evaluates input. If confidence >= 0.85, returns HTTP 403.

7. For blocked requests, the strike engine increments the IP strike count, triggering auto-ban if threshold reached.

8. Valid requests are proxy-forwarded to the backend, stripped of hop-by-hop headers, and response returned.

Project Outcomes & System Integration

Synthesizing the Subsystems:

The core engineering challenge was merging five separate development phases into a single application pipeline.

Integration outcomes include:

• Zero leakage: All traffic passes through the before_request hook. Bypass attacks are prevented.

• Low footprint: Checks are executed in order of cost, minimizing CPU and memory overhead.

• Persistent state: Config settings, log logs, and blacklist state are persisted, maintaining WAF configuration states.

INTEGRATION LIFECYCLE SUMMARY

- Phase 1 established the host Flask container blueprint context.
- Phase 2 injected the configuration, rate limiter, Basic Auth, and regex engine rules.
- Phase 3 added the LightGBM classifier prediction loop and blacklist auto-bans.
- Phase 4 placed the 7-layer file upload validator and TLS proxy.
- Phase 5 connected the reverse proxy routes, target backends, and test runner.

Final Project Assessment & Deliverables

DELIVERABLES CHECKLIST STATUS

✓ [COMPLETED] Flask scaffold, request Hooks, static layouts, templates.

✓ [COMPLETED] Twelve-Factor YAML configurations & environment overrides.

✓ [COMPLETED] Per-IP Rate Limiting (200/day, 100/hour) & HTTP Basic Auth gates.

✓ [COMPLETED] 30+ compiled regex rules for SQLi, XSS, Path Traversal, CMDi, File Injection.

✓ [COMPLETED] LightGBM classifier tf-idf model & feedback training APIs.

✓ [COMPLETED] 7-layer file scanner, SVG scripts, archive bombs, VBA macros, double extension checks.

✓ [COMPLETED] Nginx TLS Proxy termination, ProxyFix headers restoration.

✓ [COMPLETED] Transparent proxy forwarding, hop-by-hop stripping, redirect handling.

✓ [COMPLETED] Vulnerable demo backend, unit tests, integration test suite.

THANK YOU

Questions & Answers

FlaskGuard Project Defense | Technical Report

AUTHORS: Amine Saoud, Saaf Ayyoub, Abdelmoghit El Asraoui, Mohamed Ben Mouh, Hamza Amhidi